

Technical Requirements Document

Time-Off Frontend for ExampleHR

Prepared by: Umair Shahzad

Date: June 4, 2026

1. Problem Statement

The core challenge in building this Time-Off frontend is not the UI itself. The UI is straightforward. The real problem is that ExampleHR does not own the data it displays. HCM is the source of truth and HCM does not wait for our frontend to catch up.

In my experience with React applications, stale data is usually manageable because your own backend controls when data changes. But here, HCM can change balances at any moment. Work anniversaries, year-start resets, and admin corrections can all happen without telling us. Our frontend finds out only when it asks. This makes background synchronization the most critical design concern in this system, not an afterthought.

The second hard problem is trust. HCM occasionally returns a success response that is not actually successful. A frontend that takes every 200 OK at face value will show users incorrect information. We cannot afford that, especially for the manager who is making an approval decision. If a manager approves a request based on a balance that was already stale or silently wrong, the damage is not just technical. It breaks trust with the user.

So the two tensions this system must resolve are simple to state but hard to implement. First, show data fast but never show data that is confidently wrong. Second, give instant feedback on actions but never promise something HCM has not confirmed.

These two tensions shape every decision in this document.

2. Challenges

After going through the requirements carefully, I identified five challenges that need deliberate solutions. These are not edge cases. They are core behaviors the system will encounter regularly.

2.1 Background Balance Changes

This is the challenge I consider most difficult. HCM can update an employee's balance at any point during an active session. Work anniversaries and year-start resets are predictable but our frontend has no subscription or webhook to know when they happen. We only find out by asking. The tricky part is not fetching the update, it is deciding what to do with it when the user is already doing something. If an employee is mid-way through filling a request form and the

balance changes in the background, interrupting them is rude. Staying completely silent is dangerous. The system needs a middle ground.

2.2 Optimistic Updates and Rollbacks

When an employee submits a request, waiting for HCM to respond before updating the UI feels slow and untrustworthy. So we update the UI immediately and confirm with HCM in the background. The problem is HCM can reject the request. When that happens we need to roll back the UI cleanly and explain what happened without confusing the user.

2.3 Silent Failures from HCM

HCM sometimes returns 200 OK but does not actually process the request. This is the most dangerous failure mode because everything looks fine on the surface. After every mutation's `onSettled` fires, even on HTTP 201, we call `GET /balance` for that specific `employeeId` and `locationId`. If the returned balance does not match the expected value we treat the mutation as failed regardless of HTTP status. We restore the React Query snapshot captured in `onMutate`, set Zustand status to `rolled-back`, and show a silent failure error to the user. This verification fires on every write without exception.

2.4 Stale Data for Managers

When a manager opens a request to approve or deny it, the balance shown must be fresh. A manager making a decision based on cached data from ten minutes ago could approve something that should be denied. Every time a manager opens the approval view we must fetch the latest balance directly from the HCM real-time endpoint, not from cache. The stale indicator on the employee view fires on a 60 second timer based on the `lastUpdated` timestamp of the cached data. This is a client-side check and does not require a server round trip. A future contributor should not change this to a server-confirmed check as that would add unnecessary latency to every render cycle.

2.5 Per Employee Per Location Complexity

Each employee can have multiple balances across different locations. This means a single action like submitting a request affects only one specific row in the balance table. Our state management must be granular enough to update and roll back individual rows without affecting others. This granularity is achieved through query key design which is described in detail in Section 3.

3. Proposed Solution

3.1 Tech Stack

After considering the requirements carefully I decided on the following stack:

- Next.js App Router for the framework. I have worked with it before and its server components and route handlers make it straightforward to build both the frontend and the mock HCM endpoints in the same project without needing a separate backend.
- React Query for data fetching and state management. I have used React Query in production before and it fits this problem well. It gives fine grained cache control, built in optimistic update support with rollback, background refetching, and query invalidation. SWR can do some of this but React Query gives much more control over cache behavior which is exactly what this assignment needs.
- Zustand for UI state. React Query handles server state. Zustand handles client side UI state like which request form is open, which row is being edited, and pending confirmation dialogs. Keeping these two concerns separate makes the code easier to reason about.
- TypeScript for type safety. With complex data shapes like per employee per location balances, having strong types catches mistakes early.

3.2 Data Fetching Strategy and Query Key Design

The data layer works in three layers. First layer is initial hydration. When the app loads we call the batch HCM endpoint once to get all balances for the employee. This is the expensive call so we only make it once at the start. Second layer is real time reads for specific actions like manager approvals. Third layer is background reconciliation every 30 seconds.

Cache invalidation works at the individual row level because of how query keys are structured. Each location balance has its own specific key. The batch endpoint uses a separate key. When a rollback happens on Lahore, React Query's `setQueryData` targets only the Lahore key and Karachi and Dubai are completely untouched. This is the reason optimistic state is kept at the BalanceCard level and not the dashboard level.

The query key hierarchy is structured as follows:

```
// Batch endpoint - initial hydration only
['hcm', 'balances', employeeId]

// Per cell - authoritative real-time read
['hcm', 'balance', employeeId, locationId]
// Example: ['hcm', 'balance', 'emp_001', 'Lahore']
// Example: ['hcm', 'balance', 'emp_001', 'Karachi']

// Pending requests - manager view
['hcm', 'requests', 'pending']
```

This hierarchy means rolling back Lahore calls `setQueryData` on the Lahore key only. Karachi is a completely separate key and is never touched. This granularity is what makes per-row optimistic updates safe.

3.3 Optimistic Update Strategy

When an employee submits a request we immediately update the cached balance in React Query and mark the request as pending. We do not tell the user it is approved. We tell them it is submitted and being confirmed. This is an important distinction. The full lifecycle using React Query's mutation hooks is:

```
onMutate → snapshot current cache for this locationId
          → optimistically update balance in React Query cache
          → set Zustand requestStatus to 'optimistic-pending'
          → set inFlightLocations flag for this locationId

onError → restore snapshot to React Query cache
         → set Zustand requestStatus to 'rolled-back'
         → show error message to user

onSettled → call GET /balance for this employeeId + locationId
           → if freshBalance !== expectedBalance → treat as failure
           → restore snapshot, set status to 'rolled-back'
           → clear inFlightLocations flag
           → apply any held background updates
```

3.4 Silent Failure Detection

After every mutation's onSettled fires, even on HTTP 201, we call GET /balance for that specific employeeId and locationId. If the returned balance does not match the expected value we treat the mutation as failed regardless of the HTTP status code. We restore the React Query snapshot captured in onMutate, set the Zustand status to rolled-back, and show a silent failure error. This check fires on every write without exception because we cannot trust HCM's response codes.

The verification logic works like this:

```
// After POST /request returns 201 OK
const freshBalance = await GET /balance?employeeId&locationId
const expectedBalance = previousBalance - requestedDays

if (freshBalance !== expectedBalance) {
  // HCM lied. Roll back everything.
  queryClient.setQueryData(queryKey, snapshot)
  setRequestStatus('rolled-back')
  showError('silent-failure')
}
```

3.5 Background Refresh vs In Flight Action

If a background reconciliation fetch returns new data while the user has a request in flight we do not apply the background update immediately. We hold it in a `pendingBackgroundUpdate` state in Zustand using the `locationId` as the key. We apply the held update only in the `onSettled` callback after the in-flight request resolves. This way we never overwrite an optimistic update with background data mid action.

```
// Zustand store flags
inFlightLocations: Set<locationId>
pendingBackgroundUpdates: Map<locationId, Balance>

// When background refetch arrives
if (inFlightLocations.has(locationId)) {
  pendingBackgroundUpdates.set(locationId, newBalance)
} else {
  applyUpdate(locationId, newBalance)
}
```

3.6 Manager Approval Flow and Query Configuration

When a manager opens a request to approve or deny we always fetch the latest balance from the real time HCM endpoint before showing the approval view. We do not use cached data for this. The manager sees a small loading state while we fetch and then sees the fresh balance.

`ApprovalView` uses `staleTime` of 0 and `gcTime` of 0 on its balance query. `staleTime` 0 forces a fresh fetch on every mount even if cached data already exists. `gcTime` 0 discards the result immediately after the component unmounts so the next manager who opens the same request sees a fresh fetch and not a cached result from a previous session. The approve and deny buttons are disabled until this query resolves.

```
// ApprovalView query configuration
useQuery({
  queryKey: ['hcm', 'balance', employeeId, locationId],
  staleTime: 0, // always fetch fresh on mount
  gcTime: 0,    // discard on unmount, no cross-session cache
})
```

3.7 Test Harness Control

The mock HCM endpoints accept an `X-HCM-Simulate` header that allows tests to force specific behaviors without changing application code. This makes the test suite fully controllable and deterministic.

- `X-HCM-Simulate: silent_failure` — POST returns 201 OK but balance does not change

- X-HCM-Simulate: rejection — POST returns 500 with `hcm_unavailable` error
- X-HCM-Simulate: success — forces success regardless of other conditions

4. Alternatives Considered

4.1 SWR instead of React Query

SWR was the first alternative I considered since it is made by the same team behind Next.js. It handles basic stale while revalidate fetching well and is simpler to set up. However for this specific problem I needed fine grained control over optimistic updates and rollbacks. SWR's mutation API is more limited compared to React Query. When I thought about the rollback scenario where HCM silently fails and we need to revert a specific cache entry while keeping others intact, React Query's invalidation model gave me more confidence. So I went with React Query.

4.2 Pessimistic Updates instead of Optimistic

The safer approach would have been to wait for HCM to confirm before updating the UI. No rollback needed, no risk of showing wrong data. But in my opinion this creates a worse user experience for a system that employees use daily. Waiting two to three seconds for every request submission feels slow and untrustworthy. I chose optimistic updates but with a deliberately honest UI state. We never tell the user their request is approved until HCM confirms. We tell them it is pending confirmation. This way if a rollback happens it does not feel like a broken promise.

4.3 WebSockets or SSE for Real Time Updates

One way to solve the background sync problem is to have HCM push updates to the frontend using WebSockets or Server Sent Events. This would eliminate the need for polling entirely. However the assignment states that HCM exposes a REST API, not a real time subscription. Building a WebSocket layer on top of that would mean maintaining our own push infrastructure which adds complexity without a clear requirement. Polling every 30 seconds with React Query is simpler, predictable, and easy to adjust.

4.4 Redux for State Management

Redux was an option for managing both server and client state in one place. I have used Redux before and it works but for this scale of application it adds boilerplate that slows down development without meaningful benefit. React Query handling server state and Zustand handling UI state is a cleaner separation and easier for future contributors to understand.

4.5 Global Loading State instead of Granular

A simpler approach would have been to show a full page loading spinner whenever any data is being fetched. This is easy to implement but creates a jarring experience, especially during

background reconciliation. I chose granular loading states per row and per action so background fetches happen invisibly and only the affected component shows a loading indicator.

5. Test Strategy

Since this project uses agentic development, the test suite is where I demonstrate understanding of the system. Anyone can generate code. The tests are proof that I have thought through every failure mode and that future contributors cannot silently break the system.

I divided the tests into three layers. Each layer guards a different kind of regression and deliberately does not duplicate the concerns of other layers.

5.1 Storybook Interaction Tests

Storybook stories cover every meaningful UI state. The purpose of these stories is not just visual documentation. Each story has an interaction test attached that simulates user behavior and asserts the correct outcome. Storybook tests are responsible for visual and state rendering regressions only. They do not test network behavior.

The states covered are:

- Loading state when HCM has not responded yet
- Empty state when the employee has no balances
- Stale state when cached data is older than the reconciliation threshold
- Optimistic pending state when a request is submitted and waiting for HCM confirmation
- Optimistic rolled back state when HCM rejects the request and the UI reverts
- HCM rejected state when HCM returns a clear error with a reason
- HCM silently wrong state when HCM returns 200 OK but the balance did not change
- Balance refreshed mid session state when a background reconciliation returns a different balance than what is cached

5.2 Component Tests

Component tests use React Testing Library and focus on individual component behavior in isolation. These tests do not care about the network and they do not test visual rendering. They mock the data layer and test that the component renders correctly and responds to user interactions properly. Network behavior is integration tests' responsibility.

The critical component tests are:

- BalanceCard renders correct balance for given location
- BalanceCard shows stale indicator when lastUpdated is older than 60 seconds
- BalanceCard shows pending badge when request is in optimistic pending state

- BalanceCard shows rollback error when mutation fails
- RequestForm disables submit when requested days exceed balance prop
- RequestForm calls onSubmit with correct payload on submit
- ApprovalView fetches fresh balance on mount
- ApprovalView keeps approve button disabled while balance is loading
- ApprovalView shows conflict error when approve returns 409

5.3 Integration Tests

Integration tests run against MSW handlers and test the full flow from user action to HCM response to UI update. MSW is used specifically because it intercepts at the network level, meaning the full React Query, optimistic update, and rollback chain is exercised without mocking React Query itself. This makes the tests trustworthy. Integration tests do not test visual state rendering. That is Storybook's responsibility.

The critical integration tests are:

- Employee submits request, HCM confirms, balance updates correctly
- Employee submits request, HCM rejects, UI rolls back and shows error
- Employee submits request, HCM returns 200 OK but balance unchanged, UI detects silent failure and rolls back
- Background reconciliation fires while request is in flight, update is held until request settles
- Manager opens approval view, fresh balance is fetched before showing decision buttons
- Work anniversary bonus fires during session, balance updates with subtle notification

5.4 Why This Division

Storybook tests catch visual and state regressions. Component tests catch logic regressions. Integration tests catch data flow regressions. Together they cover the full matrix of things that can go wrong without duplicating effort across layers.

Component tests deliberately do not test network behavior. That is integration tests' responsibility. Integration tests deliberately do not test visual state rendering. That is Storybook's responsibility. This separation prevents overlap, keeps each layer fast, and makes failures easy to diagnose because a failing test points clearly to one layer of the system.

6. Component Tree

One of the requirements was to explain how the component tree maps to the data concerns. Here is how I structured it.

6.1 Employee View

- EmployeeDashboard is the top level component. It owns the batch fetch on initial load and passes balances down to children. It also owns the background reconciliation timer and the logic for holding background updates when a request is in flight. It is wrapped in an ErrorBoundary that catches unexpected failures and shows a recoverable error state.
- BalanceTable sits inside EmployeeDashboard. It receives the list of balances and renders one BalanceCard per location row.
- BalanceCard is a single row showing one location and its balance. It owns its own optimistic state. When the employee submits a request from this card the optimistic update happens at this level so only this row shows the pending state, not the entire table.
- RequestForm lives inside BalanceCard. It handles the form state, submission, and communicates the result back up to BalanceCard which then updates or rolls back the optimistic state.

6.2 Manager View

- ManagerDashboard is the top level component for the manager. It fetches the list of pending requests on load. It is also wrapped in an ErrorBoundary.
- RequestList sits inside ManagerDashboard and renders one RequestItem per pending request.
- RequestItem shows the request details. When a manager clicks to approve or deny it triggers a fresh real time balance fetch before showing the ApprovalView. The approve and deny buttons are disabled until that fresh fetch completes.
- ApprovalView shows the fresh balance and the approve and deny buttons. It never uses cached data. Every time it mounts it fetches fresh from the real time HCM endpoint using staleTime 0 and gcTime 0.

6.3 Why This Structure

Keeping optimistic state at the BalanceCard level means a rollback on one row does not affect other rows. Keeping the fresh fetch responsibility inside RequestItem means the manager always sees current data without the developer having to remember to add a fetch call manually. Wrapping both dashboards in ErrorBoundaries means unexpected failures degrade gracefully without crashing the entire page. The structure enforces the right behavior by default.